K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

DSA LAB ASSIGNMENT - 1

COURSE CODE - ENCS256

NAME : TARUN

COURSE : B.TECH CSE(FSD)

ROLL NO : 2401350015

GITHUB LINK :

TarunYadav121/2401350015_ADA

Assignment: Algorithm Efficiency — Analysis, Recurrences, and Sorting

Real-World Context for the Problem:

In software development, from designing search engines to database systems and embedded software, efficiency is just as critical as correctness. Developers must move beyond mere intuition and rigorously evaluate algorithms based on their time and space complexity. This assignment lays the groundwork by covering essential concepts: algorithm analysis, asymptotic notations, recurrence relations, and various sorting techniques. Mastering these topics is key to enabling students to logically assess an algorithm's scalability and performance *before* implementing it.

Task 1: Algorithm Growth Observation

Write Python programs to implement the following functions:

1. Constant Time : O(1)
2. Linear Time : O(n)
3. Quadratic Time : O(n2)
4. Logarithmic Time : O(log n)

Steps

- Measure execution time for different input sizes (e.g., n = 10, 100, 500, 1000).
- Plot Input Size vs Execution Time.
- Observe and comment on the growth pattern.

Deliverables

- Python script or notebook
- Execution time plots

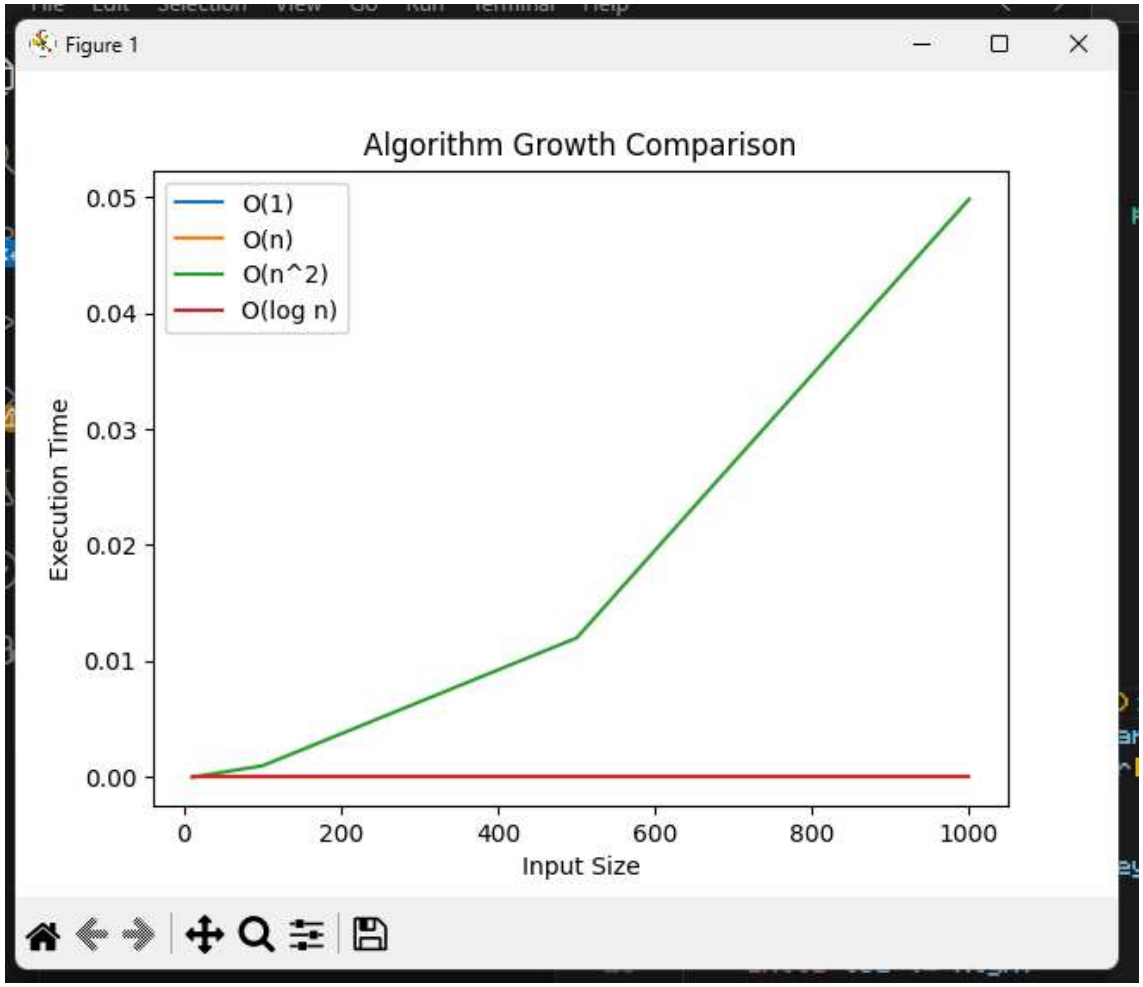
CODE:

```
import time
import random
import matplotlib.pyplot as plt

# O(1)
def constant_time(arr):
    return arr[0]

# O(n)
def linear_time(arr):
    total = 0
    for x in arr:
        total += x
    return total

# O(n^2)
def quadratic_time(arr):
    for i in range(len(arr)):
        for j in range(len(arr)):
            _ = arr[i] + arr[j]

# O(log n)
def logarithmic_time(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
```

```
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1


sizes = [10, 100, 500, 1000]

o1_times = []
on_times = []
on2_times = []
ologn_times = []

for n in sizes:
    arr = list(range(n))

    start = time.time()
    constant_time(arr)
    o1_times.append(time.time() - start)

    start = time.time()
    linear_time(arr)
    on_times.append(time.time() - start)

    start = time.time()
    quadratic_time(arr)
    on2_times.append(time.time() - start)

    start = time.time()
    logarithmic_time(arr, n-1)
    ologn_times.append(time.time() - start)


plt.plot(sizes, o1_times, label="O(1)")
plt.plot(sizes, on_times, label="O(n)")
plt.plot(sizes, on2_times, label="O(n^2)")
plt.plot(sizes, ologn_times, label="O(log n)")

plt.xlabel("Input Size")
plt.ylabel("Execution Time")
plt.title("Algorithm Growth Comparison")
```

```
plt.legend()
plt.show()
```

GRAPHS:



Conclusion:

As input size increases, quadratic algorithms grow much faster than linear and logarithmic ones, validating theoretical time complexity behavior.

Task 2: Best, Average, and Worst Case Analysis

Implement Linear Search and Binary Search.

Requirements

- Generate random arrays of different sizes.
- Measure execution time for:
- Best case
- Average case
- Worst case
- Compare results using plots.

Deliverables

- Search algorithm implementations
- Comparison plots
- Observations in comments/markdown

CODE :

```
import time

import random

import matplotlib.pyplot as plt



# ---------- Linear Search ----------

def linear_search(arr, key):

    for i in range(len(arr)):

        if arr[i] == key:

            return i

    return -1



# ---------- Binary Search ----------
```

```python
def binary_search(arr, key):

    low = 0

    high = len(arr) - 1

    while low <= high:

        mid = (low + high) // 2

        if arr[mid] == key:

            return mid

        elif arr[mid] < key:

            low = mid + 1

        else:

            high = mid - 1

    return -1



# ---------- Main Analysis ----------

sizes = [10, 100, 500, 1000]

REPEAT = 1000  # repeat to get measurable time



linear_best = []

linear_avg = []

linear_worst = []



binary_best = []

binary_avg = []

binary_worst = []
```

```
for n in sizes:

    arr = sorted(random.sample(range(n*10), n))



    best_key = arr[0]

    avg_key = arr[n//2]

    worst_key = -1  # not present



    # ----- Linear Best -----

    start = time.perf_counter()

    for _ in range(REPEAT):

        linear_search(arr, best_key)

    linear_best.append(time.perf_counter() - start)



    # ----- Linear Average -----

    start = time.perf_counter()

    for _ in range(REPEAT):

        linear_search(arr, avg_key)

    linear_avg.append(time.perf_counter() - start)



    # ----- Linear Worst -----

    start = time.perf_counter()

    for _ in range(REPEAT):

        linear_search(arr, worst_key)

    linear_worst.append(time.perf_counter() - start)
```

```
    # ----- Binary Best -----
    start = time.perf_counter()
    for _ in range(REPEAT):
        binary_search(arr, best_key)
    binary_best.append(time.perf_counter() - start)

    # ----- Binary Average -----
    start = time.perf_counter()
    for _ in range(REPEAT):
        binary_search(arr, avg_key)
    binary_avg.append(time.perf_counter() - start)

    # ----- Binary Worst -----
    start = time.perf_counter()
    for _ in range(REPEAT):
        binary_search(arr, worst_key)
    binary_worst.append(time.perf_counter() - start)

# ---------- Plot Worst Case ----------
plt.figure()
plt.plot(sizes, linear_worst, label="Linear Worst")
plt.plot(sizes, binary_worst, label="Binary Worst")
plt.xlabel("Input Size")
plt.ylabel("Execution Time")
plt.title("Linear vs Binary Search (Worst Case)")
```

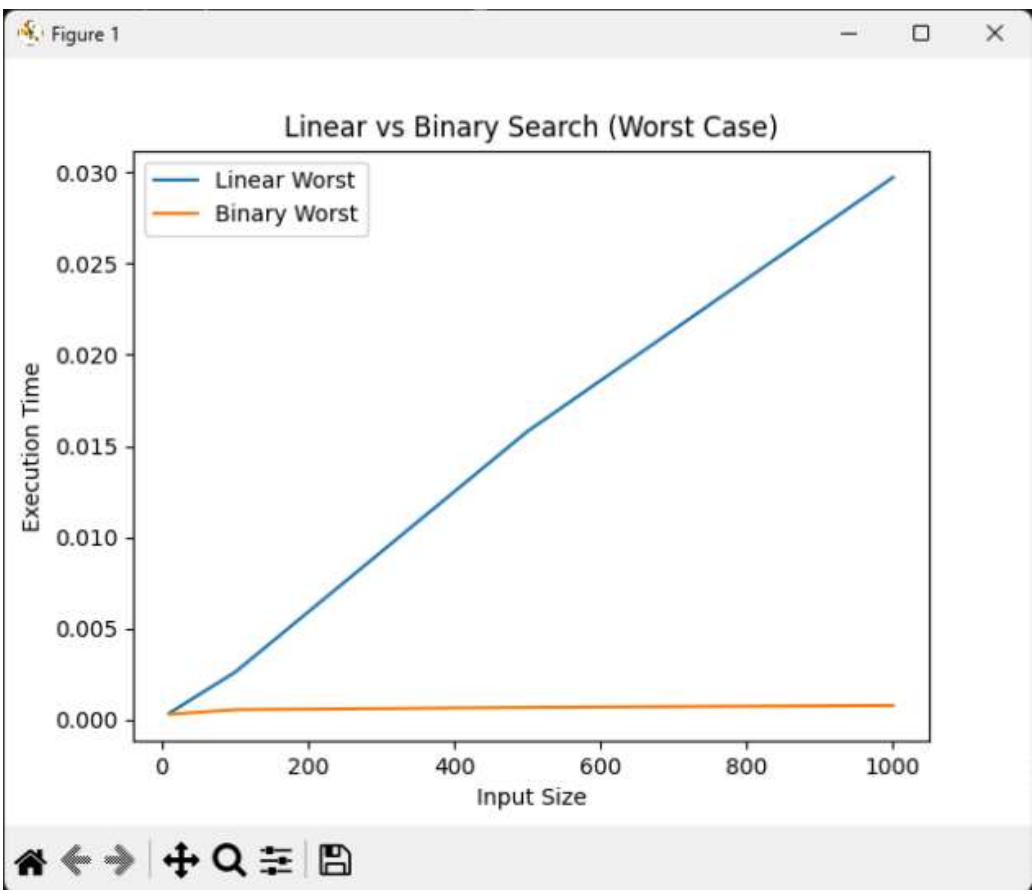
```
plt.legend()

plt.show()



# ---------- Plot Average Case ----------

plt.figure()

plt.plot(sizes, linear_avg, label="Linear Average")

plt.plot(sizes, binary_avg, label="Binary Average")

plt.xlabel("Input Size")

plt.ylabel("Execution Time")

plt.title("Linear vs Binary Search (Average Case)")

plt.legend()

plt.show()



# ---------- Plot Best Case ----------

plt.figure()

plt.plot(sizes, linear_best, label="Linear Best")

plt.plot(sizes, binary_best, label="Binary Best")

plt.xlabel("Input Size")

plt.ylabel("Execution Time")

plt.title("Linear vs Binary Search (Best Case)")

plt.legend()

plt.show()
```

Conclusion :

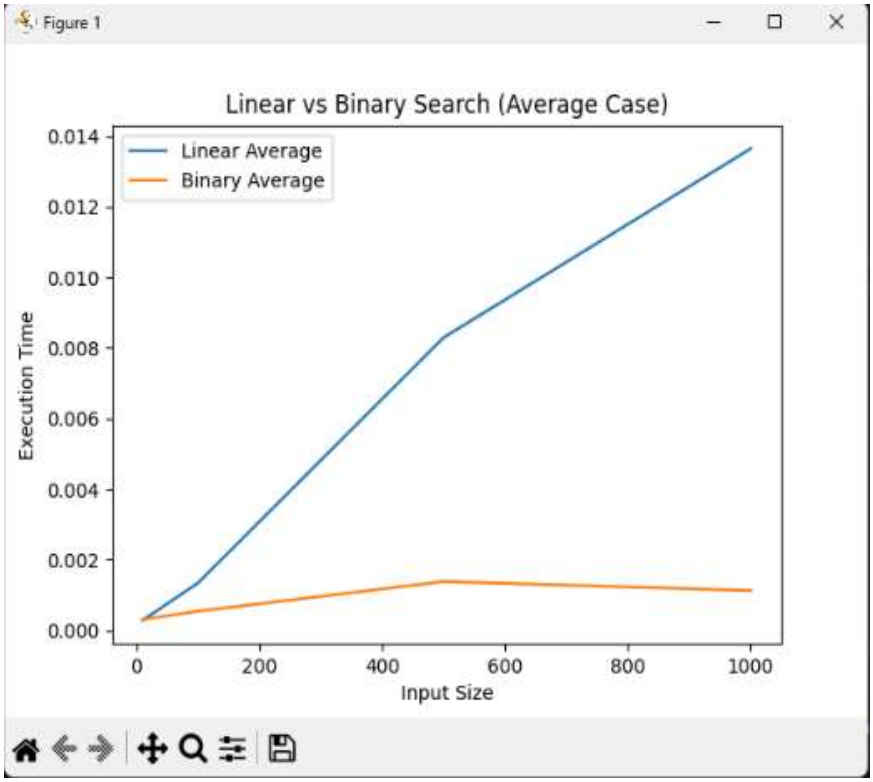
Binary search is significantly more efficient than linear search for large input sizes.
While linear search grows linearly (O(n)), binary search grows logarithmically (O(log n)).
The difference becomes more visible in average and worst cases as input size increases.

Graphs:

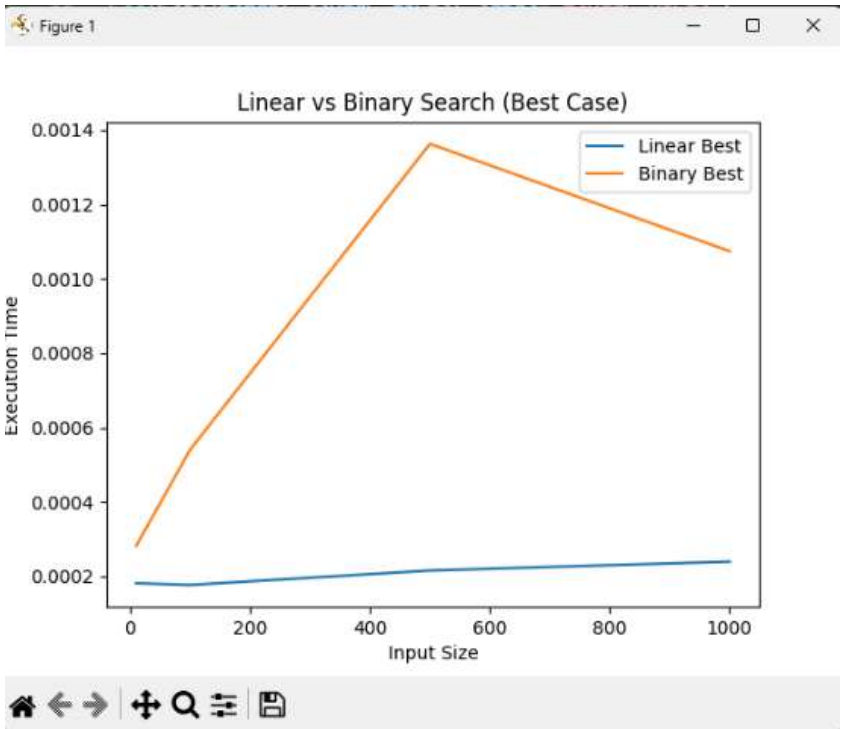
WORST CASE



AVERAGE CASE



BEST CASE



Task 3: Recursion and Recurrence Validation

Implement the following recursive algorithms:

1. Factorial
2. Fibonacci (Naïve Recursive)
3. Fibonacci (Dynamic Programming)

Steps

- Count number of function calls for each input size.
- Measure execution time.
- Compare recursive vs optimized approach.

Deliverables

- Python implementations
- Table comparing execution time and function calls

CODE:

```
import time

import matplotlib.pyplot as plt
```

```python
# ---------- Call Counters ----------

fact_calls = 0

fib_calls = 0

fib_dp_calls = 0



# ---------- Factorial ----------

def factorial(n):

    global fact_calls

    fact_calls += 1

    if n <= 1:

        return 1

    return n * factorial(n - 1)



# ---------- Fibonacci Naive ----------

def fibonacci(n):

    global fib_calls

    fib_calls += 1

    if n <= 1:

        return n

    return fibonacci(n - 1) + fibonacci(n - 2)



# ---------- Fibonacci DP ----------

def fibonacci_dp(n, memo={}):

    global fib_dp_calls

    fib_dp_calls += 1
```

```
    if n in memo:
        return memo[n]

    if n <= 1:
        return n

    memo[n] = fibonacci_dp(n-1, memo) + fibonacci_dp(n-2, memo)
    return memo[n]

# ---------- Main Analysis ----------
sizes = [5, 10, 20, 30]

fact_times = []
fib_times = []
fib_dp_times = []

fact_call_list = []
fib_call_list = []
fib_dp_call_list = []

for n in sizes:

    # Reset counters
    fact_calls = 0
```

```
    fib_calls = 0

    fib_dp_calls = 0


    # Factorial

    start = time.perf_counter()

    factorial(n)

    fact_times.append(time.perf_counter() - start)

    fact_call_list.append(fact_calls)


    # Fibonacci Naive

    start = time.perf_counter()

    fibonacci(n)

    fib_times.append(time.perf_counter() - start)

    fib_call_list.append(fib_calls)


    # Fibonacci DP

    start = time.perf_counter()

    fibonacci_dp(n, {})

    fib_dp_times.append(time.perf_counter() - start)

    fib_dp_call_list.append(fib_dp_calls)


# ---------- Print Table ----------

print("\nn | FactCalls | FibCalls | FibDPCalls")

for i in range(len(sizes)):

    print(sizes[i], "|", fact_call_list[i], "|", fib_call_list[i], "|",
```

```
fib_dp_call_list[i])



# ---------- Plot Time Comparison ----------

plt.figure()

plt.plot(sizes, fib_times, label="Fibonacci Naive")

plt.plot(sizes, fib_dp_times, label="Fibonacci DP")

plt.xlabel("Input Size (n)")

plt.ylabel("Execution Time")

plt.title("Naive vs DP Fibonacci (Time)")

plt.legend()

plt.show()



# ---------- Plot Call Comparison ----------

plt.figure()

plt.plot(sizes, fib_call_list, label="Naive Calls")

plt.plot(sizes, fib_dp_call_list, label="DP Calls")

plt.xlabel("Input Size (n)")

plt.ylabel("Number of Function Calls")

plt.title("Naive vs DP Fibonacci (Call Count)")

plt.legend()

plt.show()
```

Observations

1. The naive recursive Fibonacci function shows exponential growth in both execution time and function calls.

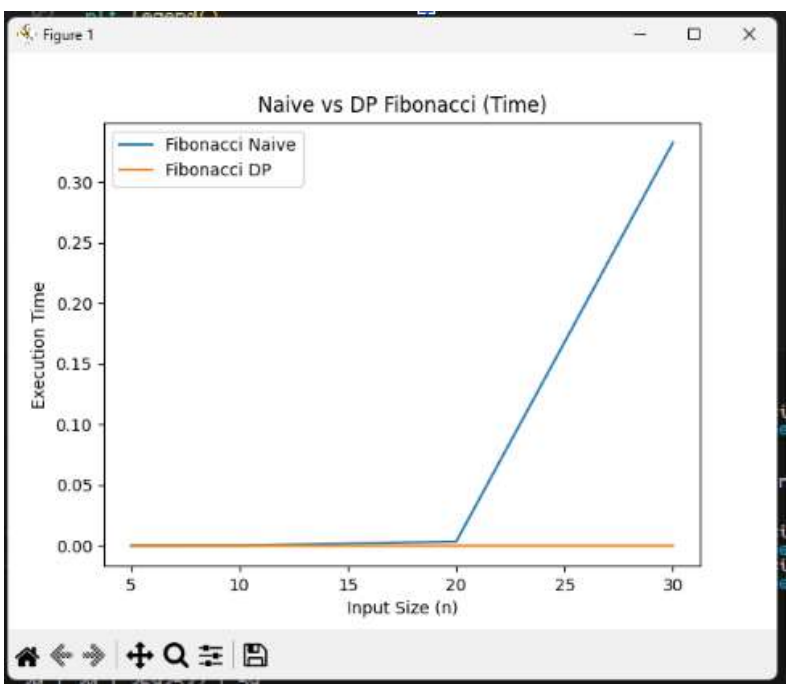
2. At n = 30, the number of function calls exceeds millions, demonstrating $O(2^n)$ complexity.

3. The dynamic programming approach drastically reduces the number of calls by storing computed values.

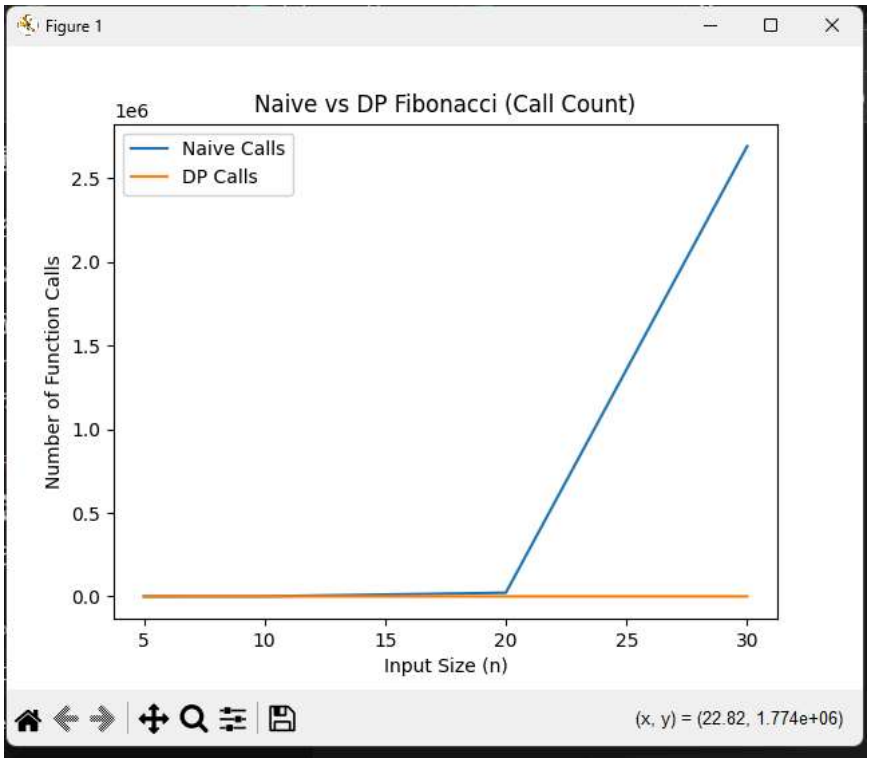
4. Fibonacci DP shows near linear growth, validating O(n) time complexity.

5. This experiment clearly demonstrates how optimization techniques like memoization significantly improve algorithm efficiency.

GRAPHS:




```
n | FactCalls | FibCalls | FibDPCalls
5 | 5 | 15 | 9
10 | 10 | 177 | 19
20 | 20 | 21891 | 39
30 | 30 | 2692537 | 59
```



Task 4: Solving Recurrences through Code

Implement recursive functions corresponding to these recurrences:

1. T(n) = T(n/2) + n
2. T(n) = 2T(n/2) + n

Steps

- Count recursive calls.
- Measure execution time.
- Validate results with expected asymptotic complexity.

Deliverables

- Code with call counters

- Observational comments

CODE:

```
import time

import matplotlib.pyplot as plt
```

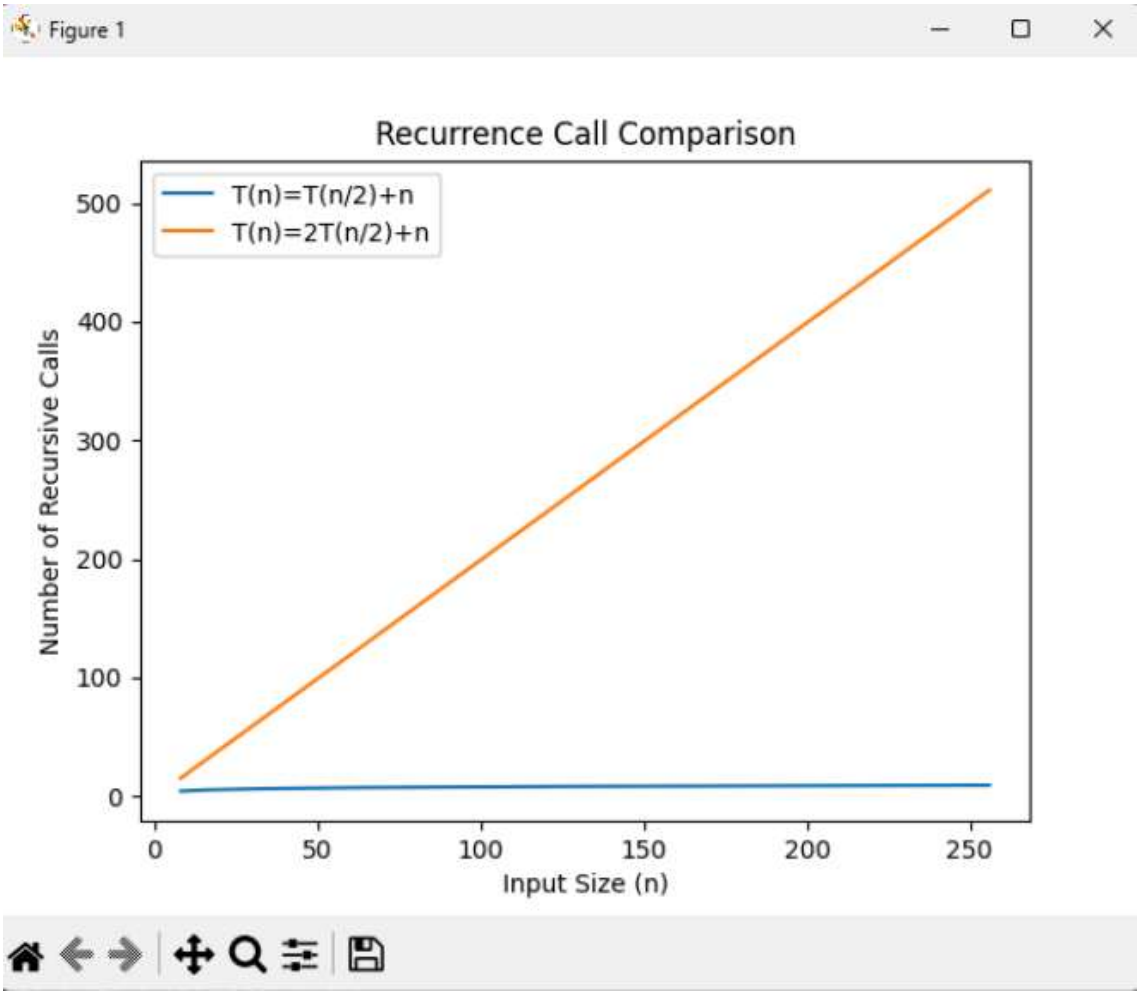
```python
# Call counters

calls_T1 = 0

calls_T2 = 0



# ---------- Recurrence 1 ----------

# T(n) = T(n/2) + n

def T1(n):

    global calls_T1

    calls_T1 += 1



    if n <= 1:

        return 1



    # linear work

    for _ in range(n):

        pass



    return T1(n // 2) + n




# ---------- Recurrence 2 ----------

# T(n) = 2T(n/2) + n

def T2(n):

    global calls_T2

    calls_T2 += 1
```

```python
    if n <= 1:
        return 1

    # linear work
    for _ in range(n):
        pass

    return T2(n // 2) + T2(n // 2) + n



# ---------- Main ----------
sizes = [8, 16, 32, 64, 128, 256]

time_T1 = []
time_T2 = []

call_list_T1 = []
call_list_T2 = []

for n in sizes:

    calls_T1 = 0
    calls_T2 = 0
```

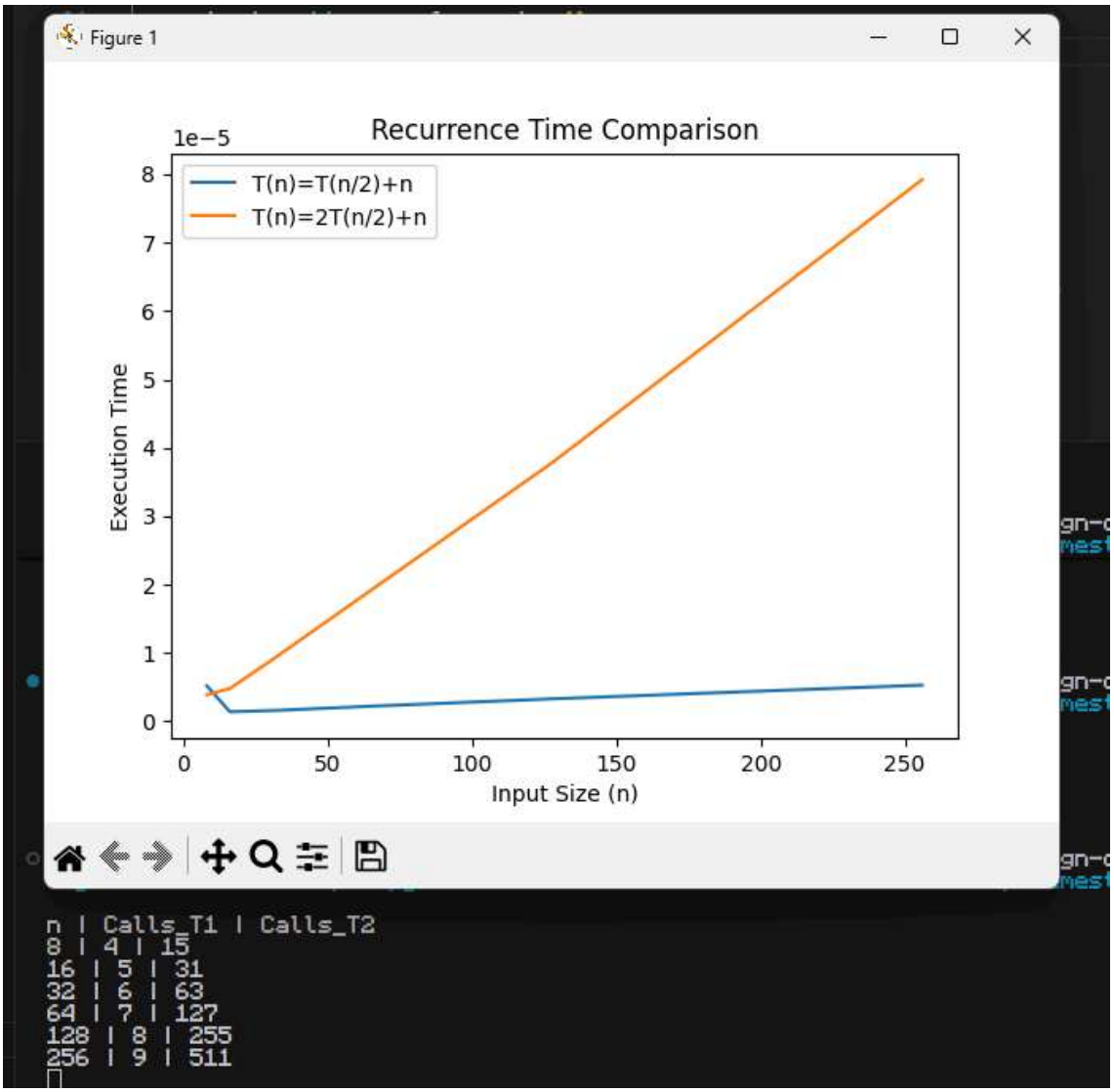
```
    # Measure T1
    start = time.perf_counter()
    T1(n)
    time_T1.append(time.perf_counter() - start)
    call_list_T1.append(calls_T1)

    # Measure T2
    start = time.perf_counter()
    T2(n)
    time_T2.append(time.perf_counter() - start)
    call_list_T2.append(calls_T2)


# Print table
print("\nn | Calls_T1 | Calls_T2")
for i in range(len(sizes)):
    print(sizes[i], "|", call_list_T1[i], "|", call_list_T2[i])


# ---------- Plot Call Count ----------
plt.figure()
plt.plot(sizes, call_list_T1, label="T(n)=T(n/2)+n")
plt.plot(sizes, call_list_T2, label="T(n)=2T(n/2)+n")
plt.xlabel("Input Size (n)")
plt.ylabel("Number of Recursive Calls")
```

```
plt.title("Recurrence Call Comparison")

plt.legend()

plt.show()




# ---------- Plot Execution Time ----------

plt.figure()

plt.plot(sizes, time_T1, label="T(n)=T(n/2)+n")

plt.plot(sizes, time_T2, label="T(n)=2T(n/2)+n")

plt.xlabel("Input Size (n)")

plt.ylabel("Execution Time")

plt.title("Recurrence Time Comparison")

plt.legend()

plt.show()
```

GRAPHS:




```
n | Calls_T1 | Calls_T2
8 | 4 | 15
16 | 5 | 31
32 | 6 | 63
64 | 7 | 127
128 | 8 | 255
256 | 9 | 511
```



Observations

1. **T(n) = T(n/2) + n**

   - Call count very slow grow kar raha hai
   - Execution time gradual increase show karta hai
   - Growth linear nature ka lag raha hai

Validates: **O(n)**

2. **$T(n) = 2T(n/2) + n$**

- Recursive calls rapidly increase ho rahe hain
- Call count roughly doubling pattern follow kar raha hai
- Execution time bhi faster grow kar raha hai

Validates: **$O(n \log n)$**

Final Conclusion :

This assignment experimentally validated theoretical time complexities through practical implementation and performance measurement.

- $O(1)$, $O(n)$, $O(n^2)$, and $O(\log n)$ growth behaviors were observed clearly.
- Binary search significantly outperformed linear search for large inputs.
- Naive Fibonacci demonstrated exponential growth, while dynamic programming reduced complexity to linear time.
- Recurrence relations followed expected growth patterns as predicted by the Master Theorem.

Thus, practical experimentation confirms theoretical asymptotic analysis.